Bynry Backend Engineering Intern Case Study

Candidate: Shrish Vats

Part 1: Code Review & Debugging

1. Identified Issues & Production Impact

- **Lack of Database Transactions (Atomicity):** The code performs two separate `db.session.commit()` calls. If the first succeeds but the server crashes before the second, a product is created without any inventory record. This leads to "ghost products" in the database.
- **Missing Input Validation:** The code assumes all keys (like `initial_quantity`) are present in the request. If a field is missing, the server will trigger a `KeyError` and crash with a 500 Internal Server Error.
- **SKU Uniqueness Vulnerability:** There is no check to see if a SKU already exists. Attempting to insert a duplicate SKU will cause a database constraint error that isn't handled gracefully.
- **Poor Error Handling:** Without a `try...except` block, any database connection glitch results in a generic error message, making it difficult for the user to understand what went wrong.

2. Corrected Version (Python/Flask)

Python

```
@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.json

    # 1. Validation: Ensure all mandatory fields are present
    required_fields = ['name', 'sku', 'price', 'warehouse_id', 'initial_quantity']
    if not all(field in data for field in required_fields):
        return {"error": "Missing required fields"}, 400

    try:
        # 2. Use a single transaction context for Atomicity
        with db.session.begin():

            # 3. Business Rule: SKUs must be unique
            existing_product = Product.query.filter_by(sku=data['sku']).first()
```

```
        if existing_product:
            return {"error": f"Product SKU {data['sku']} already exists"}, 409

        # Create Product
        product = Product(
            name=data['name'],
            sku=data['sku'],
            price=data['price']
        )
        db.session.add(product)

        # Flush to get product.id for the next step without committing yet
        db.session.flush()

        # Create Inventory Record
        inventory = Inventory(
            product_id=product.id,
            warehouse_id=data['warehouse_id'],
            quantity=data['initial_quantity']
        )
        db.session.add(inventory)

    # Success: The 'with' block automatically commits here
    return {"message": "Product and Inventory created", "product_id": product.id}, 201

except Exception as e:
    # Automatic rollback occurs if using 'with db.session.begin()'
    return {"error": "Internal server error occurred"}, 500
```

Part 2: Database Design

1. Schema Design (Relational Model)

- **Companies:** `id` (PK), `name`, `created_at`.
- **Warehouses:** `id` (PK), `company_id` (FK), `name`, `location`.
- **Products:** `id` (PK), `sku` (Unique), `name`, `price` (Decimal), `is_bundle` (Boolean).
- **Product_Bundles:** `parent_id` (FK to Products), `child_id` (FK to Products), `quantity`.
- **Inventory:** `product_id` (FK), `warehouse_id` (FK), `quantity`, `low_stock_threshold`.
- **Suppliers:** `id` (PK), `name`, `contact_email`.

2. Identified Gaps & Questions

- **Bundle Logic:** Does a bundle have its own physical stock, or is its availability calculated dynamically based on its components?
- **Supplier Mapping:** Can a single product be provided by multiple suppliers for the same company?
- **Currency Support:** Should the price field support multiple currencies for different companies?

3. Decisions & Justifications

- **Self-Referencing Bundle Table:** I chose a separate `Product_Bundles` table to allow a single "Product" entry to act as a parent for other products, supporting flexible "Combo" offers.
- **Decimal for Price:** I used the `Decimal` type instead of `Float` to prevent precision errors in financial calculations.
- **Threshold-based Inventory:** Storing the `low_stock_threshold` in the `Inventory` table allows different warehouses to have different alert levels for the same product.

Part 3: API Implementation

1. API Design: Low-Stock Alerts

Endpoint: `GET /api/companies/:company_id/alerts/low-stock`

Logic Description:

1. Filter the **Inventory** table where `quantity <= low_stock_threshold`.
2. Join with **Warehouses** to ensure the results belong to the specific `company_id`.
3. Join with **Products** and **Suppliers** to get the contact info for re-ordering.
4. **Requirement Check:** Filter for products that have at least one sale record in the last 30 days.

2. Mock Implementation (Node.js/Express)

JavaScript

```
app.get('/api/companies/:company_id/alerts/low-stock', async (req, res) => {
    const { company_id } = req.params;

    try {
        const query = `
            SELECT p.name, p.sku, i.quantity, i.threshold, s.name as supplier_name,
s.contact_email
            FROM Inventory i
            JOIN Products p ON i.product_id = p.id
            JOIN Warehouses w ON i.warehouse_id = w.id
```

```
        JOIN Suppliers s ON p.supplier_id = s.id
        WHERE w.company_id = ?
          AND i.quantity <= i.threshold
          AND p.id IN (SELECT product_id FROM Sales WHERE sale_date > NOW() -
INTERVAL 30 DAY)
      `;

      const [alerts] = await db.execute(query, [company_id]);
      res.json({ alerts, count: alerts.length });
    } catch (error) {
      res.status(500).json({ error: "Could not fetch alerts" });
    }
});
```

3. Edge Cases Handled

- **No Recent Activity:** Products with low stock but zero sales in 30 days are excluded to avoid cluttering the manager's dashboard.
- **Zero Threshold:** If a threshold is set to 0, alerts are only triggered when the product is completely out of stock.
- **Security:** The API requires a `company_id` to ensure data isolation between different clients.